Practical 1

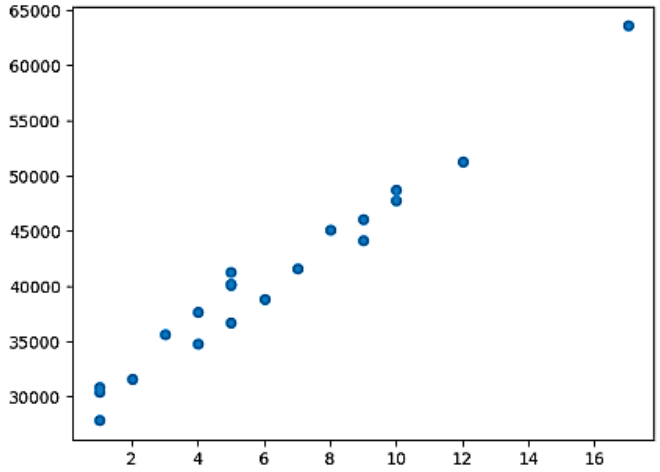
A) Design a simple machine learning model to train the training instances and test the same.

```
[In]  import pandas as pd
      import matplotlib.pyplot as plt
[In]  #Read data
      df = pd.read_csv('downloads/income_dataset.csv')
      df.head()
[Out]       age   experience   income
      0     25          1      30450
      1     30          3      35670
      2     47          2      31580
      3     32          5      40130
      4     43         10      47830
 [In] #Determine X and y
      X = df[['age','experience']]
      y = df['income']
[In]  print(X)
[Out]    age  experience
      0  25       1
      1  30       3
      2  47       2
      3  32       5
      4  43      10
      5  51       7
      6  28       5
      7  33       4
      8  37       5
      9  39       8
      10 29       1
      11 47       9
      12 54       5
      13 51       4
      14 44      12
      15 41       6
      16 58      17
      17 23       1
      18 44       9
      19 37      10
[In]  print(y)
[Out] 0   30450
      1   35670
      2   31580
      3   40130
      4   47830
      5   41630
      6   41340
      7   37650
      8   40250
      9   45150
      10  27840
```

```
11   46110
12   36720
13   34800
14   51300
15   38900
16   63600
17   30870
18   44190
19   48700
Name: income, dtype: int64
```

[In]
```
#Plot to compare Experience(Independent variable) with Income(Dependent variable)
plt.scatter(df['experience'],df['income'])
plt.show()
```

[Out]



[In]
```
#Create training set and testing set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)
```

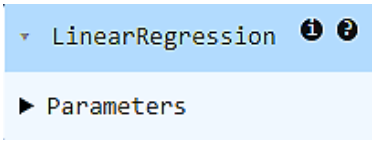
[In]
```
len(X_train)
```

[Out] 16

[In]
```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

[In]
```
#Train the model
lr.fit(X_train, y_train)
```

[Out]



[In]
```
#Use the trained model to predict the testing set
lr.predict(X_train)
```

[Out]
```
array([52638.68966758, 38521.06825445, 49116.85436964, 34968.95273512,
       34999.23295651, 31272.68933053, 39442.25573063, 31140.05494512,
       45974.1555064 , 36811.32768749, 61801.51748648, 46281.21799846,
       36781.0474661 , 40230.8088214 , 48502.72938552, 31477.39765858])
```

[In]
```
#Calculate the accuracy
lr.score(X_test, y_test)
```

[Out] 0.9373405749460351

B) Perform Data Loading, Feature selection (Principal Component analysis) and Feature Scoring and Ranking.

```
[In]  import pandas as pd
      import numpy as np
      from sklearn.preprocessing import StandardScaler
      from sklearn.datasets import load_wine
      import matplotlib.pyplot as plt
      sc = StandardScaler()
[In]  #Define PCA function
      def PCA(x, no_of_components):
        #Step 1
        Xmeaned = sc.fit_transform(x)
        #Step 2
        cov_matrix = np.cov(Xmeaned.T)
        #Step 3
        eigen_values, eigen_vectors = np.linalg.eigh(cov_matrix)
        #Step 4
        sorted_index = np.argsort(eigen_values)[::-1]
        sorted_eigen_values = eigen_values[sorted_index]
        sorted_eigen_vectors = eigen_vectors[:,sorted_index]
        selected_eigenvectors = sorted_eigen_vectors[:, :no_of_components]
        #Step 5
        x_pca = Xmeaned.dot(selected_eigenvectors)
        #Add on
        total_variance = np.sum(sorted_eigen_values)
        explained_variance_ratio = sorted_eigen_values / total_variance
        return x_pca, explained_variance_ratio[:no_of_components]
[In]  #Load data
      wine_data = load_wine(as_frame=True)
      df = wine_data.frame
[In]  df.head()
```

[Out]

	alcohol	malic_acid	ash	…….	proline	target
0	14.23	1.71	2.43	…….	1065.0	0
1	13.20	1.78	2.14	…….	1050.0	0
2	13.16	2.36	2.67	…….	1185.0	0
3	14.37	1.95	2.50	…….	1480.0	0
4	13.24	2.59	2.87	…….	735.0	0

```
[In]  df.shape
[Out] (178, 14)
[In]  #Prepare the data and target
      X = wine_data.data
      y = wine_data.target
[In]  X.head()
```

[Out]

	alcohol	malic_acid	ash	…….	proline
0	14.23	1.71	2.43	…….	1065.0
1	13.20	1.78	2.14	…….	1050.0
2	13.16	2.36	2.67	…….	1185.0
3	14.37	1.95	2.50	…….	1480.0
4	13.24	2.59	2.87	…….	735.0

```
[In]  y.head()
[Out] 0   0
      1   0
```

```
2    0
3    0
4    0
Name: target, dtype: int64
```

[In]
```
#Apply PCA function to the data
wine_PCA,pc_proportions = PCA(X,5)
```

[In]
```
#Plot the graph of new PCA data generated
markers = ["o", "s", "^"]
plt.figure(figsize=(8, 6))
for target_class, marker in zip(np.unique(y), markers):
   mask = y == target_class
   plt.scatter(wine_PCA[mask, 0], wine_PCA[mask, 1], marker=marker, color="steelblue",
      edgecolor="k", s=60, label=wine_data.target_names[target_class], )
plt.title("PCA of Wine Dataset")
plt.xlabel("1st Principal Component")
plt.ylabel("2nd Principal Component")
plt.legend(title="Classes")
plt.show()
```

[Out]



[In]
```
#Principal Components Proportions
for i, prop in enumerate(pc_proportions):
   print(f"Principal Component {i+1}: {prop:.2%} of total variance")
print(f"Total variance explained by these {len(pc_proportions)} components:
{sum(pc_proportions):.2%}")
```

[Out]
```
Principal Component 1: 39.54% of total variance
Principal Component 2: 17.84% of total variance
Principal Component 3: 10.33% of total variance
Principal Component 4: 6.63% of total variance
Principal Component 5: 6.27% of total variance
Total variance explained by these 5 components: 80.60%
```

Practical 2

A) Implement the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.

```
[In]  import pandas as pd
      import numpy as np
[In]  #Load data
      data = pd.read_csv('downloads\data.csv')
      data
```

[Out]

	sky	air temp	humidity	wind	water	forecast	enjoy sport
0	sunny	warm	normal	strong	warm	same	yes
1	sunny	warm	high	strong	warm	same	yes
2	rainy	cold	high	strong	warm	change	no
3	sunny	warm	high	strong	cool	change	yes

```
[In]  #Determine the independent variables
      concepts = np.array(data)[:,:-1]
      print(concepts)
[Out] [['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
       ['sunny' 'warm' 'high' 'strong' 'warm' 'same']
       ['rainy' 'cold' 'high' 'strong' 'warm' 'change']
       ['sunny' 'warm' 'high' 'strong' 'cool' 'change']]
[In]  #Determine target variables
      target = np.array(data)[:,-1]
      print(target)
[Out] ['yes' 'yes' 'no' 'yes']
[In]  #Find S algorithm
      def find_s_algo(ins, tar):
        specific_hypo = None
        for i,val in enumerate(tar):
          if val == 'yes':
            specific_hypo = ins[i].copy()
            break
        #print(specific_hypo)
        for i, val in enumerate(ins):
          if tar[i] == 'yes':
            for x in range(len(specific_hypo)):
              if ins[i][x] != specific_hypo[x]:
                specific_hypo[x] = '?'
          print(f"X{i+1}: {specific_hypo}")
        return specific_hypo
      hypo = find_s_algo(concepts, target)
      print("Final hypothesis: ",hypo)
[Out] X1: ['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
      X2: ['sunny' 'warm' '?' 'strong' 'warm' 'same']
      X3: ['sunny' 'warm' '?' 'strong' 'warm' 'same']
      X4: ['sunny' 'warm' '?' 'strong' '?' '?']
      Final hypothesis:  ['sunny' 'warm' '?' 'strong' '?' '?']
```

B) For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.

```
[In]  import pandas as pd
      import numpy as np
[In]  #Load data
      data = pd.read_csv('downloads\data.csv')
      data
```

[Out]

	sky	air temp	humidity	wind	water	forecast	enjoy sport
0	sunny	warm	normal	strong	warm	same	yes
1	sunny	warm	high	strong	warm	same	yes
2	rainy	cold	high	strong	warm	change	no
3	sunny	warm	high	strong	cool	change	yes

```
[In]  #Determine the independent variables
      concepts = np.array(data)[:,:-1]
      print(concepts)
[Out] [['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
       ['sunny' 'warm' 'high' 'strong' 'warm' 'same']
       ['rainy' 'cold' 'high' 'strong' 'warm' 'change']
       ['sunny' 'warm' 'high' 'strong' 'cool' 'change']]
[In]  #Determine target variables
      target = np.array(data)[:,-1]
      print(target)
[Out] ['yes' 'yes' 'no' 'yes']
[In]  #Candidate Elimination algorithm
      def candidate_elim_algo(ins, tar):
          specific_hypo = ins[0].copy()
          print("Specific hypothesis: ",specific_hypo)
          general_hypo = [["?" for i in range(len(specific_hypo))] for i in range(len(specific_hypo))]
          print("General hypothesis: ", general_hypo)

          for i, val in enumerate(ins):
              if tar[i] == 'yes':
                  for x in range(len(specific_hypo)):
                      if ins[i][x] != specific_hypo[x]:
                          specific_hypo[x] = '?'
                          general_hypo[x][x] = '?'

              if tar[i] == 'no':
                  for x in range(len(specific_hypo)):
                      if ins[i][x] != specific_hypo[x]:
                          general_hypo[x][x] = specific_hypo[x]
                      else:
                          general_hypo[x][x] = '?'

              print(f"\nS{i+1}: {specific_hypo}")
              print(f"G{i+1}: {general_hypo}")

          indices = [i for i, val in enumerate(general_hypo) if val == ['?', '?', '?', '?', '?', '?']]
          for i in indices:
              general_hypo.remove(['?', '?', '?', '?', '?', '?'])
          return specific_hypo, general_hypo
```

```
specific_final, gen_final = candidate_elim_algo(concepts, target)
print("\nFinal Specific hypothesis: ", specific_final)
print("\nFinal General hypothesis: ", gen_final)
```

[Out]
```
Specific hypothesis:  ['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
General hypothesis:  [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

S1: ['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
G1: [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

S2: ['sunny' 'warm' '?' 'strong' 'warm' 'same']
G2: [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

S3: ['sunny' 'warm' '?' 'strong' 'warm' 'same']
G3: [['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'same']]

S4: ['sunny' 'warm' '?' 'strong' '?' '?']
G4: [['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

Final Specific hypothesis:  ['sunny' 'warm' '?' 'strong' '?' '?']

Final General hypothesis:  [['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?']]
```

Practical 3

A) For a given set of training data examples stored in a .CSV file implement Least Square Regression algorithm.

[In]
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, root_mean_squared_error, r2_score
```

[In]
```
#Load data
data = pd.read_csv('test.csv')
data.head()
```

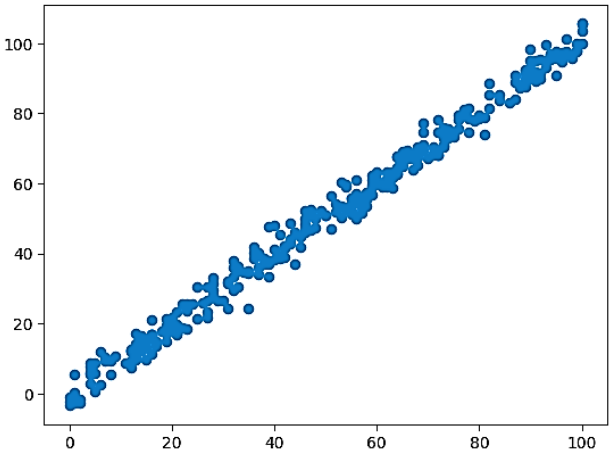
[Out]

	x	y
0	77	79.775152
1	21	23.177279
2	22	25.609262
3	20	17.857388
4	36	41.849864

[In]
```
# Preprocessing Input data
X = data.iloc[:,0]
Y = data.iloc[:,1]
```

[In]
```
plt.scatter(X,Y)
plt.show()
```

[Out]



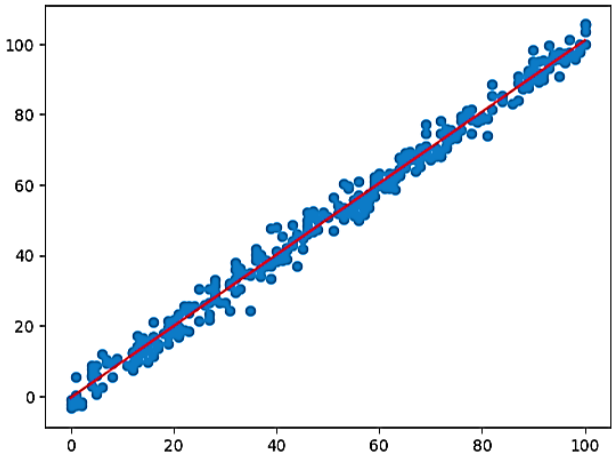
[In]
```
#Univaraite Least Square Regression
X_mean = np.mean(X)
Y_mean = np.mean(Y)
covar = 0
var = 0
for i in range(len(X)):
   covar += (X[i] - X_mean) * (Y[i] - Y_mean)
   var += (X[i] - X_mean)**2
reg_coeff = covar/var
intercept = Y_mean - reg_coeff * X_mean
print(reg_coeff, intercept)
```

[Out]
```
1.0143353551195182 -0.46181077366119894
```

[In]
```
# Making predictions
Y_pred = intercept + reg_coeff * X
```

[In]
```
plt.scatter(X,Y)
plt.plot([min(X),max(X)], [min(Y_pred),max(Y_pred)], color='red')
plt.show()
```

[Out]




```
[In]  #Evaluation Metrics
      mse = mean_squared_error(Y, Y_pred)
      rmse = root_mean_squared_error(Y, Y_pred)
      r2_val = r2_score(Y, Y_pred)
      print(f"Mean Squared Error (MSE): {mse}")
      print(f"Root Mean Squared Error (RMSE): {rmse}")
      print(f"R-squared (R^2): {r2_val}")
[Out] Mean Squared Error (MSE): 9.164287860614266
      Root Mean Squared Error (RMSE): 3.0272574817174482
      R-squared (R^2): 0.9891203611402716
[In]  from sklearn.metrics import mean_absolute_error
      mae = mean_absolute_error(Y, Y_pred)
      print(f"Mean Absolute Error (MAE): {mae}")
[Out] Mean Absolute Error (MAE): 2.410524352689129
[In]  n = len(Y)
      p = 1
      rss = np.sum((Y - Y_pred) ** 2)
      rse_sklearn = np.sqrt(rss / (n - p - 1))
      print(f"Residual Standard Error: {rse_sklearn}")
[Out] Residual Standard Error: 3.0373990762277843
```

B) For a given set of training data examples stored in a .CSV file implement Logistic Regression algorithm.

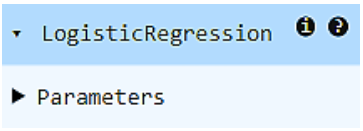
```
[In]  import pandas as pd
      import numpy as np
[In]  #Load data
      data = pd.read_csv('car_data.csv')
      data.head()
```

[Out]

	User ID	Gender	Age	AnnualSalary	Purchased
0	385	Male	35	20000	0
1	681	Male	40	43500	0
2	353	Male	49	74000	0
3	895	Male	40	107500	1
4	661	Male	25	79000	0

```
[In]  data = data.drop('Gender', axis=1)
      data.head()
```

```
[Out]       User ID     Age  AnnualSalary  Purchased
        0       385      35         20000          0
        1       681      40         43500          0
        2       353      49         74000          0
        3       895      40        107500          1
        4       661      25         79000          0
[In]    X = data.iloc[:,:-1]
        print(X)
[Out]       User ID  Age  AnnualSalary
        0       385   35         20000
        1       681   40         43500
        2       353   49         74000
        3       895   40        107500
        ..      ...  ...           ...
        996     800   47         23500
        997     407   28        138500
        998     299   48        134000
        999     687   44         73500
        [1000 rows x 3 columns]
[In]    y = data.iloc[:,-1]
        print(y)
[Out]   0      0
        1      0
        2      0
        3      1
               ..
        996    0
        997    1
        998    1
        999    0
        Name: Purchased, Length: 1000, dtype: int64
[In]    from sklearn.model_selection import train_test_split
        X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.25)
[In]    from sklearn.linear_model import LogisticRegression
        model = LogisticRegression()
[In]    #Training the model
        model.fit(X_train, y_train)
[Out]
```




```
[In]    #Prediction on the test set
        y_pred = model.predict(X_test)
        y_pred
[Out]   array([1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1,
        1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0,
        0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0,
        1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0,
        0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0,
        0, 0, 0, 1, 0, 0, 0])
[In]    #Calculating the accuracy
```

```
        accuracy = model.score(X_test, y_test)
        print(f"accuracy: {accuracy}")
[Out]   accuracy: 0.832
[In]    #Evalution Metrics
        from sklearn.metrics import confusion_matrix,precision_score, recall_score
[In]    cm = confusion_matrix(y_test, y_pred)
        cm
[Out]   array([[141,  10],
               [ 32,  67]])
[In]    precision = precision_score(y_test, y_pred)
        print(f"Precision: {precision}")
[Out]   Precision: 0.8701298701298701
[In]    recall = recall_score(y_test, y_pred)
        print(f"Recall: {recall}")
[Out]   Recall: 0.6767676767676768
```

Practical 4

A) Implement the hierarchical clustering algorithm.

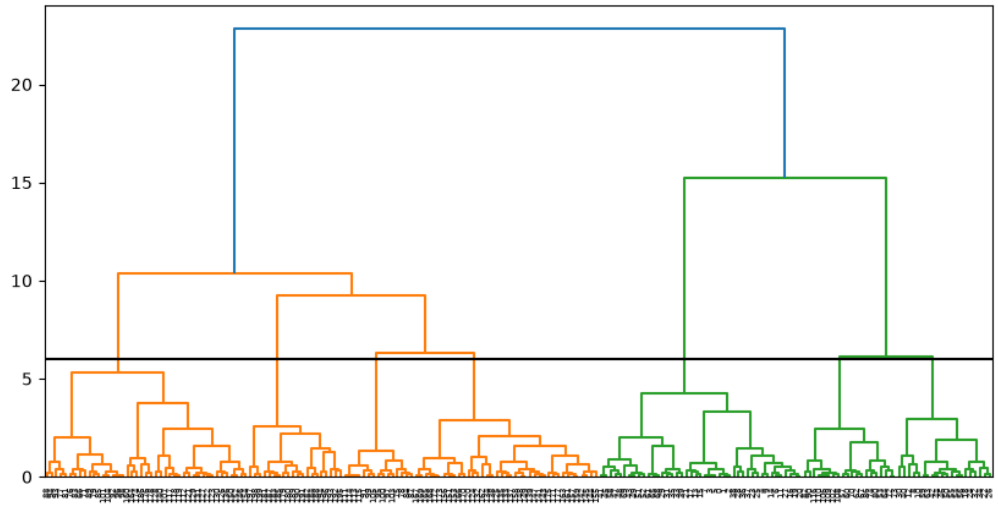
[In]
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
```

[In]
```
data = pd.read_csv("Mall_Customers.csv")
data = data.iloc[:, [0, 2, 3]].values
```

[In]
```
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(data)
```

[In]
```
#use Scipy's linkage() to form clusters and plot them with dendrogram()
linked = linkage(X_scaled, method="ward")
plt.figure(figsize=(10,5))
dendrogram(linked)
plt.axhline(y=6, c='k')
plt.show()
```

[Out]

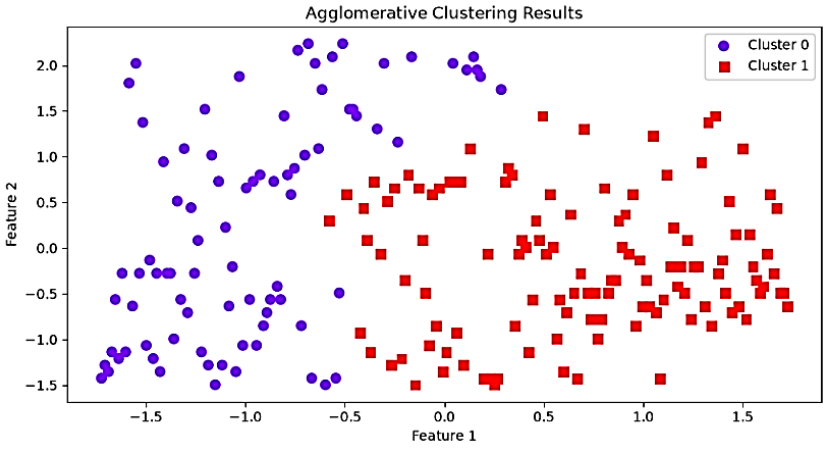


[In]
```
#perform Agglomerative Clustering
model = AgglomerativeClustering(linkage="ward")
labels = model.fit_predict(X_scaled)
```

[In]
```
print(labels)
```

[Out]
```
[0 0 0 
```

```
        ax1.legend()
        plt.show()
```
[Out]



B) Implement the K means clustering algorithm.

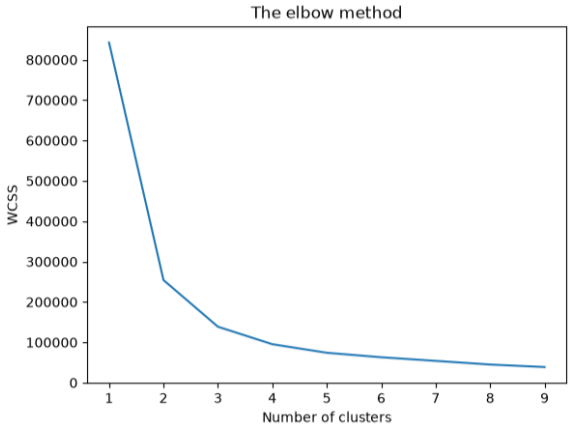
```
[In]    import warnings
        warnings.filterwarnings("ignore")
[In]    import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
[In]    df = pd.read_csv("Mall_Customers.csv")
        print(df.head())
```

[Out]

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
[In]    x = df.iloc[:, [0, 2, 3]].values
        x
[Out]   array([[  1,  19,  15],
               [  2,  21,  15],
               [  3,  20,  16],
                ..............
               [200,  30, 137]])
[In]    # Elbow method
        from sklearn.cluster import KMeans
        wcss = []
        K = range(1,10)
        for i in K:
          kmeans = KMeans(n_clusters = i, n_init = 10, random_state = 0).fit(x)
          wcss.append(kmeans.inertia_)
[In]    plt.plot(range(1, 10), wcss)
        plt.title('The elbow method')
        plt.xlabel('Number of clusters')
        plt.ylabel('WCSS')
        plt.show()
```

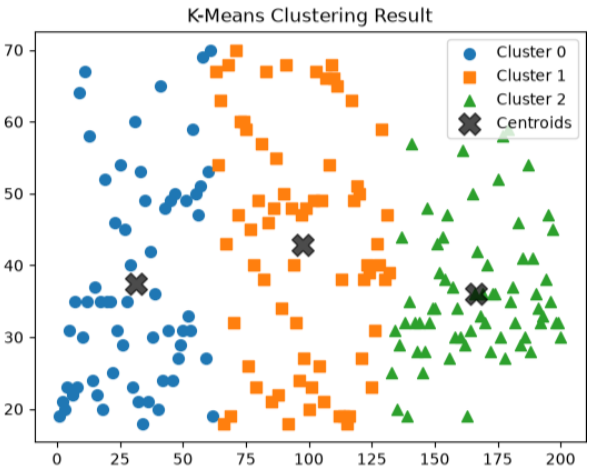
[Out]



[In]
```
kmeans = KMeans(n_clusters = 3, n_init = 10, random_state = 0)
y = kmeans.fit_predict(x)
```

[In]
```
#Visualising the clusters
centroids = kmeans.cluster_centers_
unique_labels = np.unique(y)
markers = ["o", "s", "^", "D"]
for idx, label in enumerate(unique_labels):
   cluster_mask = y == label
   plt.scatter(x[cluster_mask, 0], x[cluster_mask, 1], marker=markers[idx % len(markers)],
      s=50, label=f"Cluster {label}", )
plt.scatter(centroids[:, 0], centroids[:, 1], c="black", s=200, alpha=0.7, marker="X",
   label="Centroids",)
plt.title("K-Means Clustering Result")
plt.legend()
plt.show()
```

[Out]



Practical 5

A) Write a program to implement k-Nearest Neighbor algorithm to classify the heart_disease dataset.

```
[In]  import pandas as pd
      import numpy as np
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn import preprocessing
      from sklearn.model_selection import train_test_split
[In]  df = pd.read_csv("heart_disease.csv")
      df.columns
[Out] Index(['Unnamed: 0', 'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak',
      'slope', 'ca', 'thal', 'target'], dtype='object')
[In]  df.head()
```

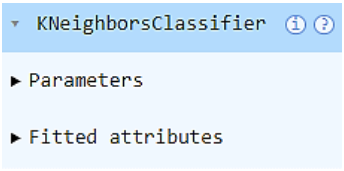
[Out]

	Unnamed: 0	age	sex	cp	trestbps	chol	fbs	restecg	thalach
0	0	63	1	3	145	233	1	0	150
1	1	37	1	2	130	250	0	1	187
2	2	41	0	1	130	204	0	0	172
3	3	56	1	1	120	236	0	1	178
4	4	57	0	0	120	354	0	1	163

	exang	oldpeak		slope	ca	thal	target
0	2.3	0	0	1	1		
0	3.5	0	0	2	1		
0	1.4	2	0	2	1		
0	0.8	2	0	2	1		
1	0.6	2	0	2	1		

```
[In]  #Define the independent variable vector X and dependent variable Y
      X = df.iloc[:,:14]
      Y = df.iloc[:,-1]
      Y.head()
[Out] 0  1
      1  1
      2  1
      3  1
      4  1
      Name: target, dtype: int64
[In]  #Preprocess the data
      X = preprocessing.StandardScaler().fit_transform(X)
[In]  Y.value_counts()
[Out] target
      1  165
      0  138
      Name: count, dtype: int64
[In]  X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size=0.2)
[In]  Y_test.shape
[Out] (61, )
[In]  #Build the KNN model for k=6 and train it
      knn = KNeighborsClassifier(n_neighbors=6)
      knn.fit(X_train, Y_train)
```

[Out]




```
[In]  #Use the KNN model for predicting the class of the test set
      Y_pred = knn.predict(X_test)
      Y_pred
[Out] array([1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0,
      1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0])
[In]  #Constructing the confusion matrix
      from sklearn.metrics import confusion_matrix
      cm = confusion_matrix(Y_test.values, Y_pred)
      cm
[Out] array([[29,  3],
           [ 2, 27]])
[In]  #Visualization of the output
      cm1 = pd.DataFrame(data = cm, index=['Yes','No'], columns=['Yes','No'])
      cm1
[Out]       Yes   No
      Yes   29    3
      No    2     27
[In]  #Calculate the accuracy of the model
      from sklearn.metrics import accuracy_score
      print(accuracy_score(Y_test, Y_pred))
[Out] 0.9180327868852459
```

B) Implement the Rule based method and test the same.

```
[In]  pip install mlxtend
[In]  pip install apyori
[In]  import pandas as pd
      import numpy as np
      import matplotlib.pyplot as plt
[In]  data = pd.read_csv("Groceries.csv")
      data.head()
```

[Out]

	Member_number	Date	itemDescription
0	1808	21-07-2015	tropical fruit
1	2552	05-01-2015	whole milk
2	2300	19-09-2015	pip fruit
3	1187	12-12-2015	other vegetables
4	3037	01-02-2015	whole milk

```
[In]  # Combine items in the same transaction into one place
      transactions = []
      for i in data.Member_number.unique():
        list_trans = list(set(data[data.Member_number == i]["itemDescription"]))
        if len(list_trans) > 0:
          transactions.append(list_trans)
      print(transactions)
[Out] [['tropical fruit', 'long life bakery product', 'whole milk', 'citrus fruit', 'semi-finished bread',
      'rolls/buns', 'meat', 'napkins', 'sugar', 'candy'], ……..]
```

[In] #use One Hot Encode the data to meet the requirement of the Apriori module given by mlxtend library
import mlxtend
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import association_rules, apriori

[In] trans_encoding = TransactionEncoder()
df2 = trans_encoding.fit(transactions).transform(transactions)
df3 = pd.DataFrame(df2, columns=trans_encoding.columns_)

[In] df3.head()

[Out]

	Instant food products	UHT-milk	…….	yogurt zwieback
0	False	False	…….	False
1	False	False	…….	False
2	False	False	…….	False
3	False	False	…….	False
4	False	False	…….	False

5 rows × 167 columns

[In] #See frequent sets formed
frequent_set = apriori(df3, min_support = 0.1, use_colnames = True)
frequent_set

[Out]

	support	itemsets
0	0.119548	(beef)
1	0.158799	(bottled beer)
2	0.213699	(bottled water)
3	0.135967	(brown bread)
4	0.126475	(butter)

…….

[In] #Find association rules
rules = association_rules(frequent_set, metric = 'lift', min_threshold = 1.1)
rules

[Out]

	antecedents	consequents	antecedent support	….	kulczynski
0	(whole milk)	(bottled water)	0.458184	….	0.385526
1	(bottled water)	(whole milk)	0.213699	….	0.385526
2	(rolls/buns)	(other vegetables)	0.349666	….	0.404654
3	(other vegetables)	(rolls/buns)	0.376603	….	0.404654
4	(whole milk)	(other vegetables)	0.458184	….	0.462934
5	(other vegetables)	(whole milk)	0.376603	….	0.462934
6	(yogurt)	(other vegetables)	0.282966	….	0.372343
7	(other vegetables)	(yogurt)	0.376603	….	0.372343
8	(rolls/buns)	(whole milk)	0.349666	….	0.450168
9	(whole milk)	(rolls/buns)	0.458184	….	0.450168
10	(yogurt)	(rolls/buns)	0.282966	….	0.355944
11	(rolls/buns)	(yogurt)	0.349666	….	0.355944
12	(sausage)	(whole milk)	0.206003	….	0.376393
13	(whole milk)	(sausage)	0.458184	….	0.376393
14	(yogurt)	(whole milk)	0.282966	….	0.430426
15	(whole milk)	(yogurt)	0.458184	….	0.430426

Practical 6

Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
[In]  import warnings
      warnings.filterwarnings('ignore')
[In]  import pandas as pd
      import numpy as np
      import math
[In]  df_tennis = pd.read_csv('PlayTennis.csv')
      df_tennis.head()
```

[Out]

	PlayTennis	Outlook	Temperature	Humidity	Wind
0	No	Sunny	Hot	High	Weak
1	No	Sunny	Hot	High	Strong
2	Yes	Overcast	Hot	High	Weak
3	Yes	Rain	Mild	High	Weak
4	Yes	Rain	Cool	Normal	Weak

```
[In]  #Function to calculate the entropy of probaility of observations
      def entropy(probs):
        import math
        return sum([-prob*math.log(prob,2)for prob in probs])
[In]  #Function to calculate the entropy of the given Data Sets with respect to target attributes
      def entropy_of_list(alist):
        from collections import Counter
        cnt=Counter(x for x in alist)
        num_instances=len(alist)
        probs = [x / num_instances for x in cnt.values()]
        return entropy(probs)
[In]  #Calculating the Entropy of the target variable
      total_entropy=entropy_of_list(df_tennis['PlayTennis'])
      print('The total Entropy is: ',total_entropy)
[Out] The total Entropy is:  0.9402859586706309
[In]  #Calculating Information Gain
      def information_gain(df, split_attribute_name, target_attribute_name, trace=0):
        df_split = df.groupby(split_attribute_name)
        nobs = len(df.index) * 1.0
        df_agg_ent = df_split.agg({target_attribute_name : [entropy_of_list, lambda x: len(x)/nobs]
      })[target_attribute_name]
        df_agg_ent.columns = ['Entropy', 'PropObservations']
        new_entropy = sum( df_agg_ent['Entropy'] * df_agg_ent['PropObservations'] )
        old_entropy = entropy_of_list(df[target_attribute_name])
        return old_entropy - new_entropy
      print('Info-gain for Outlook is :'+str( information_gain(df_tennis, 'Outlook', 'PlayTennis')))
      print('Info-gain for Humidity is: ' + str( information_gain(df_tennis, 'Humidity', 'PlayTennis')))
      print('Info-gain for Wind is:' + str( information_gain(df_tennis, 'Wind', 'PlayTennis')))
      print('Info-gain for Temperature is:' + str( information_gain(df_tennis,
      'Temperature','PlayTennis')))
[Out] Info-gain for Outlook is :0.2467498197744391
      Info-gain for Humidity is: 0.15183550136234136
      Info-gain for Wind is:0.04812703040826927
      Info-gain for Temperature is:0.029222565658954647
```

```
[In]  #ID3 Algorithm
      def id3(df, target_attribute_name, attribute_names, default_class=None):
        from collections import Counter
        cnt = Counter(x for x in df[target_attribute_name])
        if len(cnt) == 1:
          return next(iter(cnt))
        elif df.empty or (not attribute_names):
          return default_class
        else:
          default_class = max(cnt.keys())
          gainz = [information_gain(df, attr, target_attribute_name) for attr in attribute_names] #
          index_of_max = gainz.index(max(gainz))
          best_attr = attribute_names[index_of_max]
          tree = {best_attr:{}}
          remaining_attribute_names = [i for i in attribute_names if i != best_attr]
          for attr_val, data_subset in df.groupby(best_attr):
            subtree = id3(data_subset,
                    target_attribute_name,
                    remaining_attribute_names,
                    default_class)
            tree[best_attr][attr_val] = subtree
          return tree
[In]  # Get Predictor Names (all but 'class')
      attribute_names = list(df_tennis.columns)
      print("List of Attributes:", attribute_names)
      attribute_names.remove('PlayTennis')
      print("Predicting Attributes:", attribute_names)
[Out] List of Attributes: ['PlayTennis', 'Outlook', 'Temperature', 'Humidity', 'Wind']
      Predicting Attributes: ['Outlook', 'Temperature', 'Humidity', 'Wind']
[In]  #Run Algorithm
      from pprint import pprint
      tree=id3(df_tennis,'PlayTennis',attribute_names)
      print("The resulatnt DECISION TREE is: ")
      pprint(tree)
[Out] The resulatnt DECISION TREE is:
      {'Outlook': {'Overcast': 'Yes',
                   'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}},
                   'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}}}
[In]  #Result for new instance
      def classify(instance, tree, default=None): # Instance of Play Tennis with Predicted
        attribute = next(iter(tree))
        if instance[attribute] in tree[attribute].keys():
          result = tree[attribute][instance[attribute]]
          if isinstance(result, dict):
            return classify(instance, result)
          else:
            return result
        else:
          return default
[In]  df_tennis['predicted'] = df_tennis.apply(classify, axis=1, args=(tree,'No') )
      print('Accuracy is: ' + str( sum(df_tennis['PlayTennis']==df_tennis['predicted'] ) /
      (1.0*len(df_tennis.index)) ))
```

```
        df_tennis[['PlayTennis', 'predicted']]
[Out]   Accuracy is: 1.0
            PlayTennis    predicted
        0       No            No
        1       No            No
        2       Yes           Yes
        3       Yes           Yes
        4       Yes           Yes
        5       No            No
        6       Yes           Yes
        7       No            No
        8       Yes           Yes
        9       Yes           Yes
        10      Yes           Yes
        11      Yes           Yes
        12      Yes           Yes
        13      No            No
[In]    #Prediction on new Data
        training_data = df_tennis.iloc[1:-4]
        test_data  = df_tennis.iloc[-4:]
        train_tree = id3(training_data, 'PlayTennis', attribute_names)
        test_data['predicted2'] = test_data.apply(classify, axis=1, args=(train_tree,'Yes') )
        print(test_data['predicted'])
        print ('Accuracy is : ' + str( sum(test_data['PlayTennis']==test_data['predicted2'] ) /
        (1.0*len(test_data.index)) ))
[Out]   10   Yes
        11   Yes
        12   Yes
        13    No
        Name: predicted, dtype: object
        Accuracy is : 0.75
```

Practical 7

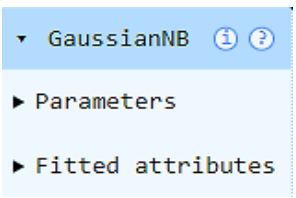
A) Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

[In]
```
import pandas as pd
import numpy as np
```
[In]
```
#Load dataset
hr_data = pd.read_csv('HR_data.csv')
hr_data.head()
```
[Out]

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend_company	Work_accident	left	promotion_last_5years	Department	salary
0	0.38	0.53	2	157	3	0	1	0	sales	low
1	0.80	0.86	5	262	6	0	1	0	sales	medium
2	0.11	0.88	7	272	4	0	1	0	sales	medium
3	0.72	0.87	5	223	5	0	1	0	sales	low
4	0.37	0.52	2	159	3	0	1	0	sales	low

[In]
```
#Checking the dimensions of the dataset
hr_data.shape
```
[Out] (14999,10)

[In]
```
X = hr_data.iloc[:,:-2]
y = hr_data.iloc[:,-1]
print(y.head)
```
[Out]
```
<bound method NDFrame.head of
0        low
1     medium
2     medium
      ...
14996    low
14997    low
14998    low
Name: salary, Length: 14999, dtype: object>
```
[In]
```
#splitting the dataset into train and test data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2)
```
[In]
```
#Import Gaussian Naive Bayes model
from sklearn.naive_bayes import GaussianNB
#Create a Gaussian Classifier
gnb = GaussianNB()
```
[In]
```
#Train the model using the training sets
gnb.fit(X_train, y_train)
```
[Out]



[In]
```
#Predict the response for test dataset
```

```
        y_pred = gnb.predict(X_test)
        print(y_pred)
[Out]   ['low' 'low' 'low' ... 'low' 'low' 'low']
[In]    #Import scikit-learn metrics module for accuracy calculation
        from sklearn.metrics import accuracy_score
        print("Accuracy: ", accuracy_score(y_test, y_pred))
[Out]   Accuracy:  0.47633333333333333
[In]    #confusion matrix
        from sklearn.metrics import confusion_matrix
        cm = np.array(confusion_matrix(y_test, y_pred))
        cm
[Out]   array([[ 28,  218,    7],
               [ 35, 1365,   20],
               [ 67, 1224,   36]])
```

B) Assuming a set of documents that need to be classified, use the Naïve Bayesian Classifier model to perform this task.

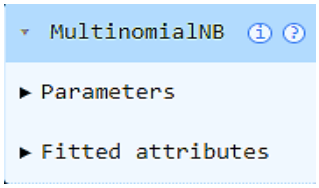
```
[In]    import pandas as pd
        import numpy as np
[In]    data = pd.read_csv('synthetic_text_data.csv')
        data.head()
```

	text	label
0	Artificial intelligence is advancing in health...	Technology
1	Football fans are excited about the upcoming W...	Sports
2	New policies regarding climate change have spa...	Politics
3	The latest blockbuster movie has shattered box...	Entertainment
4	Quantum computing promises to revolutionize in...	Technology

```
[In]    data.shape
[Out]   (85,2)
[In]    unique_labels = data['label'].unique()
        print(unique_labels)
[Out]   ['Technology' 'Sports' 'Politics' 'Entertainment']
[In]    X = data['text']
        y = data['label']
[In]    #splitting the dataset into train and test data
        from sklearn.model_selection import train_test_split
        X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2)
[In]    from sklearn.feature_extraction.text import CountVectorizer
        vectorizer = CountVectorizer()
        X_train_vectorized = vectorizer.fit_transform(X_train)
        X_test_vectorized = vectorizer.transform(X_test)
[Out]   The words or Tokens in the text documents
         ['about' 'across' 'action' 'activities' 'adapt' 'advances'  'advertisements' 'after' 'ai' 'aid''airwaves'
        'and' 'announced'  'anticipation' 'approach' 'approaching' 'are' 'areas' 'arena'  'artificial' 'as'
        'assistants' 'athletes' 'attendance' 'attracting' ……]
[In]    # Training Naive Bayes (NB) classifier on training data
        from sklearn.naive_bayes import MultinomialNB
        model = MultinomialNB()
```

```
model.fit(X_train_vectorized, y_train)
```

[Out]




```
[In]  y_pred = model.predict(X_test_vectorized)
      print(y_pred)
[Out] ['Politics' 'Technology' 'Politics' 'Politics' 'Technology' 'Technology' 'Entertainment'
      'Entertainment' 'Politics' 'Technology' 'Sports'  'Politics' 'Technology' 'Sports' 'Politics'
      'Entertainment' 'Technology']
[In]  #Calculating accuracy
      from sklearn.metrics import accuracy_score
      print("Accuracy: ", accuracy_score(y_test, y_pred))
[Out] Accuracy: 0.7647058823529411
[In]  #Confusion matrix
      from sklearn.metrics import confusion_matrix
      cm = np.array(confusion_matrix(y_test, y_pred))
      cm
[Out] array([[3, 0, 0, 2],
          [0, 6, 0, 2],
          [0, 0, 2, 0],
          [0, 0, 0, 2]])
[In]  from sklearn.metrics import recall_score
      print('\n The value of Recall' , recall_score(y_test,y_pred,average='macro'))
[Out] The value of Recall 0.8375
[In]  #Prediction on Unseen Data
      user_input = ("I love artificial intelligence and machine learning")
      user_input_vectorized = vectorizer.transform([user_input])
      predicted_label = model.predict(user_input_vectorized)
      print(f"The input text belongs to the '{predicted_label[0]}' category.")
[Out] The input text belongs to the 'Technology' category.
```

Practical 8

Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using the standard Heart Disease Data Set.

```
[In]  pip install pgmpy
[In]  import numpy as np
      import csv
      import pandas as pd
      from pgmpy.models import DiscreteBayesianNetwork
      from pgmpy.estimators import MaximumLikelihoodEstimator
      from pgmpy.inference import VariableElimination
[In]  #read Cleveland Heart Disease data
      heartDisease = pd.read_csv('heart diseases dataset.csv')
      heartDisease = heartDisease.replace('?',np.nan)
[In]  #display the data
      print('Few examples from the dataset are given below')
      print(heartDisease.head())
[Out] Few examples from the dataset are given below
        age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
      0  63    1   3       145   233    1        0      150      0      2.3      0   0     1       1
      1  37    1   2       130   250    0        1      187      0      3.5      0   0     2       1
      2  41    0   1       130   204    0        0      172      0      1.4      2   0     2       1
      3  56    1   1       120   236    0        1      178      0      0.8      2   0     2       1
      4  57    0   0       120   354    0        1      163      1      0.6      2   0     2       1
[In]  #Model Bayesian Network
      model=DiscreteBayesianNetwork([('age','target'),('sex','target'),
                     ('exang','target'),('cp','target'),
                     ('target','restecg'),('target','chol')])
[In]  #Learning CPDs using Maximum Likelihood Estimators
      print('\n Learning CPD using Maximum likelihood estimators')
      model.fit(heartDisease)
[Out] Learning CPD using Maximum likelihood estimators
      <pgmpy.models.DiscreteBayesianNetwork.DiscreteBayesianNetwork at 0x192f227f770>
[In]  # Inferencing with Bayesian Network
      print('\n Inferencing with Bayesian Network:')
      HeartDisease_infer = VariableElimination(model)
[Out] Inferencing with Bayesian Network:
[In]  #computing the Probability of RestEcg given HeartDisease is present
      print('\n 1. Probability of HeartDisease given RestEcg')
      q=HeartDisease_infer.query(variables=['restecg'],evidence ={'target':1}, joint=False)
      print(q['restecg'])
[Out]  1. Probability of HeartDisease given RestEcg
      +----------------+-----------------+
      | restecg        |  phi(restecg) |
      +==========+===========+
      | restecg(0)    |      0.4121    |
      +----------------+-----------------+
      | restecg(1)    |      0.5818    |
      +----------------+-----------------+
      | restecg(2)    |      0.0061    |
      +----------------+-----------------+
[In]  #computing the Probability of HeartDisease given Chestpain
```

```
print('\n 2. Probability of HeartDisease given Chestpain')
q=HeartDisease_infer.query(variables=['target'],evidence ={'cp':3}, joint=False)
print(q['target'])
```

[Out]

```
2. Probability of HeartDisease given Chestpain
+---------------+-----------------+
| target        |   phi(target) |
+===========+===========+
| target(0)     |      0.4588   |
+---------------+-----------------+
| target(1)     |      0.5412   |
+---------------+-----------------+
```

Practical 9

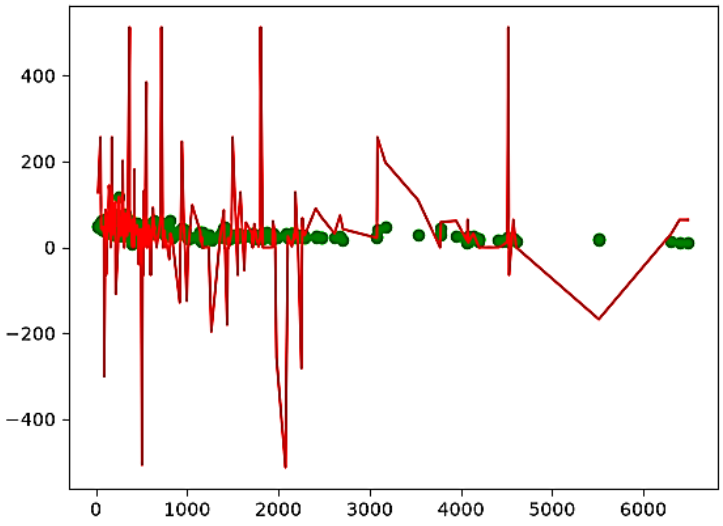
Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select the appropriate data set for your experiment and draw graphs.

```
[In]  import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
[In]  df = pd.read_csv("Real estate.csv")
      df.head()
```

[Out]

	No	transaction_date	house_age	distance_to_the_nearest_MRT_station	number_of_convenience_stores	latitude	longitude	house_price
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.500	13.3	561.98450	5	24.98746	121.54391	54.8
4	5	2012.833	5.0	390.56840	5	24.97937	121.54245	43.1

```
[In]  def kernel(point, xmat, k):
        m,n = np.shape(xmat)
        weights = np.matrix(np.eye((m)))
        for j in range(m):
          diff = point - xmat[j]
          weights[j,j] = np.exp(diff*diff.T/(-2.0*k**2))
        return weights
[In]  def localWeight(point, xmat, ymat, k):
        wei = kernel(point,xmat,k)
        W = (xmat.T*(wei*xmat)).I*(xmat.T*(wei*ymat.T))
        return W
[In]  def localWeightRegression(xmat, ymat, k):
        m,n = np.shape(xmat)
        ypred = np.zeros(m)
        for i in range(m):
          ypred[i] = xmat[i]*localWeight(xmat[i],xmat,ymat,k)
        return ypred
[In]  dist = np.array(df.distance_to_the_nearest_MRT_station)
      price = np.array(df.house_price)
[In]  mdist = np.matrix(dist)
      mprice = np.matrix(price)
[In]  m = np.shape(mdist)[1]
      one = np.matrix(np.ones(m))
      X = np.hstack((one.T,mdist.T))
[In]  ypred = localWeightRegression(X,mprice,0.4)
      SortIndex = X[:,1].argsort(0)
      xsort = X[SortIndex][:,0]
[In]  fig = plt.figure()
      ax = fig.add_subplot(1,1,1)
      ax.scatter(dist, price, color='green')
      x_plot = np.array(xsort[:, 1]).reshape(-1)
      y_plot = np.array(ypred[SortIndex]).reshape(-1)
      ax.plot(x_plot, y_plot, color='red', linewidth=1.5)
      plt.show()
[Out]
```


400
200
0
−200
−400
0
1000
2000
3000
4000
5000
6000

Practical 10

A) Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets

```
[In]  import numpy as np
[In]  X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
      y = np.array([[0], [1], [1], [0]])
      X = X/np.amax(X,axis =0)
      y = y/100
[In]  epoch = 50000
      lr = 0.4
      inputlayer = 2
      hiddenlayer = 3
      outputlayer = 1
[In]  wh = np.random.uniform(size=(inputlayer,hiddenlayer))
      bh = np.random.uniform(size=(1,hiddenlayer))
      wout = np.random.uniform(size=(hiddenlayer,outputlayer))
      bout = np.random.uniform(size=(1,outputlayer))
[In]  def sigmoid(x):
        return 1 / (1 + np.exp(-x))

      def sigmoid_derivative(x):
        return x * (1 - x)
[In]  for i in range(epoch):
        #Forward
        hinp=np.dot(X,wh) + bh
        hlayer_act = sigmoid(hinp)
        outinp= np.dot(hlayer_act,wout)+bout
        output = sigmoid(outinp)

        #Backward
        EO = y-output
        outgrad = sigmoid_derivative(output)
        d_output = EO * outgrad
        EH = d_output.dot(wout.T)
        hiddengrad = sigmoid_derivative(hlayer_act)
        d_hiddenlayer = EH * hiddengrad

        wout += hlayer_act.T.dot(d_output) *lr
        wh += X.T.dot(d_hiddenlayer) *lr

      print("Input: \n" + str(X))
      print("Actual Output: \n" + str(y))
      print("Predicted Output: \n" ,output)
[Out] Input:
      [[0. 0.]
       [0. 1.]
       [1. 0.]
       [1. 1.]]
      Actual Output:
      [[0.  ]
```

[0.01]
[0.01]
[0.]]
Predicted Output:
[[0.00716048]
[0.00341466]
[0.0044604]
[0.00236165]]